SEGMENTATION-BASED HEATMAPS AS POST-HOC EXPLAINABILITY METHOD

KAJ SOELLAART (12738549)
Amsterdam School of Economics, University of Amsterdam
27/06/2024

SUPERVISOR: BRAM WOUTERS

This thesis explores segmentation-based heatmaps as a post-hoc explainability method for addressing the Black Box problem in computer vision. By integrating a segmentation model with a Convolutional Neural Network (CNN), the thesis introduces Segmentation-based CNNs (SCNNs) and Frozen Segmentation-based CNNs (FSCNNs). The FSCNN model can create heatmaps to explain the decisions of the CNN in a post-hoc manner. The results indicate that while the CNN outperforms the SCNN in test accuracy, the FSCNN model achieves slightly lower accuracy but provides enhanced interpretability. Segmentation-based heatmaps seem to improve upon some traditional methods in terms of interpretability and computational efficiency. While promising, further research is needed to improve the segmentation-based heatmaps and to address the Black Box problem in the best possible manner.

Kaj Soellaart (12738549) : 12738549@uva.nl

Statement of Originality

This document is written by Student Kaj Soellaart who declares to take full responsibility for the contents of this document. I declare that the text and the work presented in this document are original and that no sources other than those mentioned in the text and its references have been used in creating it. UvA Economics and Business is responsible solely for the supervision of completion of the work and submission, not for the contents.

1. LITERATURE REVIEW

The architecture of a Deep Neural Network (DNN), a popular type of AI model, is associated with the so-called Black Box problem. This problem arises from the near impossibility for humans to understand the model's transformation from input data to output (Rudin et al. (2022)). This lack of understanding is at the root of possible problems, and stands in the way of societal trust in these models (Beisbart and Räz (2022)). Goodman and Flaxman (2017) have shown that under the European Union's General Data Protection Regulation (GDPR), a person that is subjected to the decision of a machine is granted the right to an explanation. Providing this explanation requires a deeper understanding of complex models.

This thesis proposes segmentation-based heatmaps to enhance understanding of AI models used in the domain of computer vision. This section provides the necessary context to understand the problem and the proposed solution. The subsequent sections assess the technical aspects of the segmentation-based heatmaps.

1.1. *The Black Box Problem*

The Black Box problem is not a one-dimensional issue and it might even "be more appropriate to speak of many Black Box Problems–one for every stakeholder" (Zednik (2021)). Each stakeholder group interacts with AI systems differently and has distinct needs for explanations. For example, end users may want straightforward explanations about how decisions impacting them are made. In contrast, regulatory authorities may require an understanding of data processing techniques to verify compliance with legislation.

Zednik (2021) proposes structuring explanations to address different levels of understanding: the computational level, which answers *what* the system is doing and *why*; the algorithmic level, which deals with *how* the system processes inputs to produce outputs; and the implementational level, which explains *where* within the system the processing occurs. According to Zednik, explaining a model involves acquiring knowledge at one or more of these levels. Therefore, Zednik's proposed framework aims to provide a much-needed structured approach to thinking about explanations in AI. This framework is used to assess the proposed explainability method in the Results section.

To tackle the Black Box problem, a distinction can be made between Explainable AI (XAI) and Interpretable AI. The former uses post-hoc methods to unravel what an AI model does, while the latter builds the model a priori to enhance understanding (Rudin et al. (2022)). Despite the differences between these approaches, the terms and their contexts are often conflated. This conflation illustrates the lack of a conceptual foundation in the domains of interpretability and XAI (Beisbart and Räz (2022)).

1.2. *Convolutional Neural Networks*

A Convolutional Neural Network (CNN) is a type of Deep Neural Network (DNN) designed for recognizing patterns in visual data, such as images. CNNs extract hierarchical features through convolutional layers, non-linear activation functions like ReLU, pooling layers, and fully connected layers (O'shea and Nash (2015)).

Inspired by the visual cortex of animals, CNNs use local receptive fields to detect patterns in small segments of an image. Convolutional layers apply learnable filters to produce feature maps highlighting edges, textures, and shapes. Images are typically formatted in three dimensions: height, width, and depth, with depth corresponding to color channels.

Non-linear activations introduce complexity, and pooling layers reduce spatial dimensions to help achieve spatial invariance and reduce computational complexity. Fully connected layers then transform the two-dimensional feature maps into a one-dimensional vector for classification or regression tasks.

CNNs are trained end-to-end using gradient-based optimization methods like stochastic gradient descent (SGD) with backpropagation, adjusting filter weights to minimize the loss function. CNNs are highly effective for image-related tasks, capturing spatial hierarchies and complex features for image classification, object detection, and segmentation.

1.3. *Post-Hoc Explainability Methods for CNNs*

Multiple post-hoc methods have been described to tackle the problem of limited understanding of CNNs. Nielsen et al. (2022) distinguish between perturbation-based and gradient-based methods. Both methods aim to estimate the salience of input features for the output, which can be visualized using heatmaps (Nielsen et al. (2022)). LIME is an example of a perturbation-based method, which perturbs the input values around a specific area of the image and then assesses the corresponding changes in the model's predictions (Ribeiro et al. (2016)). Cian et al. (2020) concluded that LIME is not suitable for all AI models and suggested that the choice of post-hoc method should depend on the model and the user. Nielsen et al. (2022) have noted that perturbation-based attribution are computationally intensive because the salience of all pixels have to be assessed.

Gradient-based methods calculate gradients of the output with respect to the input through backpropagation (Nielsen et al. (2022)). This allows for the computation of attribution (salience) scores for the input pixels, showing the importance of individual pixels for the model's prediction. Using these scores, an attribution map (heatmap) can be created to visually highlight the most salient input parts for the model's output.

However, perturbation-based and gradient-based attribution maps are not the definitive solution to the Black Box problem. These maps are known to be noisy, often assigning high attribution scores to irrelevant features (Nielsen et al. (2022)). Despite enhancements using various techniques, the model continues to exhibit several limitations. Nielsen et al. (2022) describe multiple requirements for the explainability of attribution methods: sensitivity, saturation, implementation invariance, input invariance, fidelity, and adversarial robustness.

Besides the general requirements for explainability methods, visual analysis of attribution maps may be the best way to assess the quality of explanations for human operators (Nielsen et al. (2022)). This makes sense because, in the end, explanations must align with the human way of interpreting visual data, which is through subjective visual analysis. The pitfalls of relying solely on the visual 'fitness' of an attribution map have been demonstrated by (Adebayo et al. (2020)). In conclusion, heatmaps are a popular post-hoc explainability technique that can be evaluated using several metrics.

1.4. *Image Segmentation in computer vision*

Image segmentation is a fundamental activity in computer vision and it is crucial for many applications, such as medical image analysis. The main objective of image segmentation is to create separate, homogeneous sections from an image based on parameters like color, texture, or intensity. This method makes it easier to recognize boundaries (lines, curves, etc.) and objects in pictures.

Minaee et al. (2020) have written a thorough analysis of the developments in image segmentation methods. They demonstrate how more sophisticated algorithms that make use of deep neural networks have replaced more conventional methods to provide outcomes that are more precise and effective.

A major contribution of deep learning to image segmentation has been the development of Fully Convolutional Networks (FCNs). Introduced by Long et al. (2015), FCNs retain spatial information throughout the network, which allows them to produce segmentation maps that correspond in size to the input images. This capability is partly achieved by replacing fully connected layers with convolutional layers.

2. RESEARCH DESIGN

This section elaborates on the methodological considerations for this research. First, the mathematical definitions for the classification model are spelled out. This is the model that the post-hoc explainability method aims to make more interpretable. Secondly, an extensive overview on the mathematics for the *segmenter* part of the SCNN is given. Thirdly, the manner in which the segmentation-based heatmaps are created is explained. Afterwards, the training considerations are detailed. Lastly, the dataset which is used to train the models is presented.

2.1. *Classification Model*

The classification model in this research is a CNN, about which the general features have been detailed in Section 1.2. The mathematical function that represents the classification model can be expressed as follows: $f(x) : \mathbb{R}^{C \times H \times W} \rightarrow [0, 1]^K$, where an image $x \in \mathbb{R}^{C \times H \times W}$ is assigned to K probabilities, one for each of the possible K classes. Using the *argmax* function, the class with the highest probability is chosen to estimate the correct class. In this formulation, C stands for the input channels, which is 3 for RGB colored images. Note that H and W stand for the height and width of the image. Also be aware that this mathematical formulation illustrates the process a single image takes through the CNN network. In practice, multiple images pass though the network simultaneously which would add the dimension of the batch size, B. This addition also applies to the formulation of the segmenter in the following section.

The exact architecture of the CNN classification model is provided in the Appendix.

2.2. *Segmenter Model*

For the segmenter, we define a segment classifier $h : \mathbb{R}^{C\times H\times W} \to [0,1]^{d\times Q}$. The classifier uses an image $x \in \mathbb{R}^{C\times H\times W}$ as input. The output is, for each of the d pixels, a probability distribution over Q possible segments. Mathematically expressed it looks like $[h_q(x)]_i \in [0,1]$, where this represents the probability that pixel $i \in \{1,2,\ldots,d\}$ is a member of segment $q \in \{1,2,\ldots,Q\}$. In this research the segmenter is a CNN itself, which can be considered to be a FCN as described in the literature review. This means the segmenter mainly consists out of fully-connected layers. The exact architecture of the CNN classification model can be found in the appendix.

Using the segment classifier h, a segment average $\bar{x}_q$ for each segment q is computed as:

$$\bar{x}_q^c = \frac{\sum_{i=1}^{d} [h_q(x)]_i x_i^c}{\sum_{j=1}^{d} [h_q(x)]_j}$$

Where:

- x_i^c represents the value of the i-th pixel in channel c (one of Red, Green, or Blue).
- $[h_q(x)]_i$ is the probability that the i-th pixel belongs to segment q.
- $\bar{x}_q^c$ is the average value of channel c for segment q.
- $\bar{x}_q$ is a vector $(\bar{x}_q^R, \bar{x}_q^G, \bar{x}_q^B)$ for RGB images.

The formula above calculates the weighted average of the pixel values within each segment, where the weights are given by the probability that each pixel belongs to the segment.

The final segmented image $\tilde{x} = g(x)$ is then constructed as:

$$\tilde{x}_i = [g(x)]_i = \sum_q [h_q(x)]_i \bar{x}_q$$

The segmenter, which transforms an image into a segmented image of the same dimensions, is therefore defined by g with the property of $\mathbb{R}^{C\times H\times W} \to \mathbb{R}^{C\times H\times W}$. The segmented image $g(x)$ retains the same spatial dimensions as the original image x, but the pixel values are modified to highlight different segments within the image. The classification function f then takes the segmented image $g(x)$ as input and outputs a probability distribution over the K classes.

$$(f \circ g)(x) = f(g(x)) \in [0,1]^K$$

Here, $f(g(x))$ represents the final output of the Segmentation-based CNN (SCNN), where each of the K elements in the output vector corresponds to the probability of the image belonging to one of the K classes.

2.3. *Segmentation-based heatmaps*

The proposed post-hoc explainability method is a segmentation-based heatmap. This section describes the way these segmentation-based salience maps are created.

Let $z(x) = f(g(x))$ be the function representing the SCNN's output for input image x. First, the original output and the class of interest, c_i, are calculated for each image x_i:

$$\hat{y}_i = z(x_i), \qquad c_i = \arg\max \hat{y}_i$$

$$\hat{y}_{i,c_i} = \hat{y}_i[c_i]$$

Next, for each segment q, the perturbed image is computed by adding a small perturbation ϵ to the segment:

$$x_i^{(q)} = x_i + \epsilon \cdot \mathbb{1}_{\{\text{seg}(x_i)=q\}}$$

where $\mathbb{1}_{\{\text{seg}(x_i)=q\}}$ is the indicator function that is 1 for pixels belonging to segment q and 0 otherwise.

Then, the perturbed output and the corresponding score are computed:

$$\hat{y}_i^{(q)} = f(x_i^{(q)}), \qquad \hat{y}_{i,c_i}^{(q)} = \hat{y}_i^{(q)}[c_i]$$

Finally, the salience for segment q is calculated as:

$$\text{Salience}(i, q) = \frac{|\hat{y}_{i,c_i}^{(q)} - \hat{y}_{i,c_i}|}{\epsilon}$$

In this way the salience of an all segments within an image can be calculated and visualised.

2.4. *Training Considerations*

In this research, an important distinction need to be made during the training phase. An important choice in this thesis is the amount of segments, Q, which is set to 5. This means the segmenter can assign every pixel in the image to one of 5 segments.

Simultaneously training the two models (segmenter and CNN) changes the classification model's (CNN) weights and, as a result, its behavior. The model in which this simultaneous training has been taken place, will be referred to as the 'SCNN' model. This model can't serve as a post-hoc explainability tool as it changes the parameters of the original classification model. However, analyzing the SCNN is valuable for understanding how the segmenter affects accuracy and interpretability when both models are trained together.

Since the segmenter is crucial to create the segmentation-based heatmaps, special attention is required during the training phase of both the segmenter and the classification model. To ensure that the segmenter does not alter the weights of the classification model, parameter freezing is applied. This technique allows the segmenter to be trained for optimal accuracy while keeping the classification model's parameters constant. The model that employs a pre-trained CNN with frozen parameters is referred to as the Frozen Segmentation-based CNN (FSCNN). This FSCNN enables the creation of the segmentation-based heatmap which is a post-hoc explainability method, as it preserves the original model's parameters and aims to explain its workings. The quantitative and visual results of the segmenter are discussed in the first section of the Results.

2.5. *Dataset*

The CNN used in this study is designed to effectively classify images from the CIFAR-10 dataset. CIFAR-10 consists of 60,000 color images (32x32 pixels) across ten classes, each containing 6,000 images. This means that both H and W are equal to 32, and C is equal to 3 as the pixels in the image have a red, blue and green value. The training set consist of 50,000 images, and the remaining 10,000 images are used for testing. The CIFAR-10 dataset is used often in academic research, which enhances the comparison of this research with others.

3. RESULTS

This section presents the research findings on implementing and evaluating the segmentation-based heatmap as a post-hoc explainability method.

First, the outcomes of the training procedures for the CNN, SCNN, and FSCNN models are detailed. The aim is to analyze how the segmenter reacts to different training environments.

Second, a visual analysis of the segmented images produced by the SCNN and FSCNN is conducted. This analysis illustrates the model's ability to divide input images into meaningful segments, providing insights into its effectiveness as an explainability tool. Key visual patterns and segment quality are highlighted and discussed.

Third, the produced segment sizes are examined. This analysis focuses on the distribution and consistency of segment sizes generated by the SCNN and FSCNN.

Finally, the segmentation-based salience maps are presented and analyzed. This includes a visual discussion of the salience maps and a quantitative analysis of the standard deviation of the segments and computational efficiency. Additionally, the philosophical implications are discussed using the framework by Zednik (2021).

3.1. *Training of the models*

This section discusses the results for the training and test accuracy of the CNN, SCNN and FSCNN model. As the CNN and the SCNN model update their parameters, one epoch at a time, the training accuracy is expected to converge to 1. This is because the model, after being trained for enough epochs, gets close to 'memorizing' all the train data. The test accuracy is expected to converge to a certain value, where a higher value means better accuracy on the test data. As can be seen from Figure 1, this holds for both the CNN and the SCNN model.

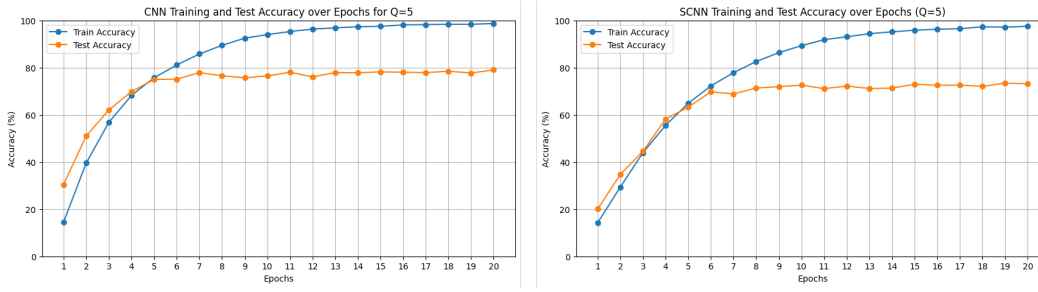

FIGURE 1.—Training and test accuracy scores for the CNN (left), SCNN (right).

Both models show similar behavior during the training phase. The CNN's initial increase for the training and test accuracy is higher, as can be seen from Table 1. After multiple epochs, the training accuracy of both models converge to 1, as expected.

However, there is a pronounced difference in the overall performance between the CNN and the SCNN for the classification task on the test data. This difference becomes around 5-6 % when both models have been trained more than 10 epochs, where the CNN model performs better.

Epoch	CNN		SCNN	
	Train Accuracy (%)	Test Accuracy (%)	Train Accuracy (%)	Test Accuracy (%)
1	14.53	30.44	14.26	20.06
2	39.55	51.05	29.38	34.75
3	56.82	62.02	43.95	44.56
5	75.68	74.98	64.88	63.27
7	85.79	77.89	77.85	68.86
10	94.00	76.48	89.30	72.57
20	98.67	79.05	97.53	73.14

TABLE I

TRAINING AND TEST ACCURACY FOR CNN AND SCNN AT SPECIFIC EPOCHS

Figure 2 makes clear that the convergence of the training accuracy to 100 % does not take place for the FSCNN model. This result is in line with the expectation, as the parameters of the CNN part are frozen. The increase in accuracy is therefore only to attributed to the updating of the segmenter's parameters. After around 3 epochs, the increase in training and test accuracy seem to be minor. The convergence of the FSCNN is quicker than for the CNN and SCNN model.

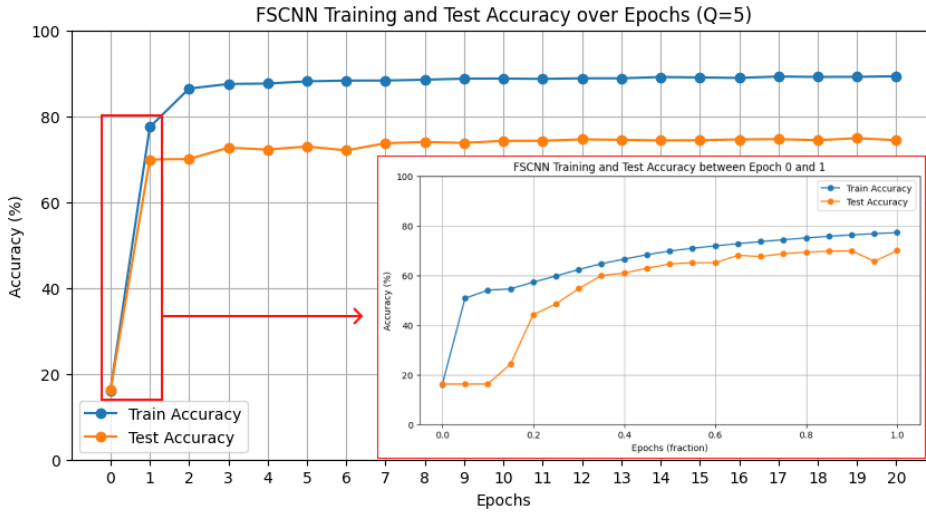

FIGURE 2.—Training and test accuracy scores the FSCNN and outlined in red an additional graph showing the accuracies between epoch 0-1.

The red outlined graph within Figure 2 shows the training and test accuracies between epoch 0 and 1. The second observation (0.05 in the graph) corresponds to the processing of a single mini-batch (128 images). For readability of the graph, subsequent observations are taken at intervals of 20 mini-batches. This means that the subsequent observations include 21, 41, and 61 mini-batches. This interval-based evaluation provides detailed snapshots of the model's performance throughout the first epoch.

Note that the graph does not perfectly align with the completion of one full epoch because the total number of mini-batches is 381, while a full epoch consists of approximately 391 mini-batches (50,000 test images divided into batches of 128). Despite this, the graph provides valuable insight into the early learning phase of the FSCNN.

Figure 2 and Table 2 make clear that the FSCNN model improves rapidly during the first 121 mini-batches. Afterwards, this increase in both training and test accuracy becomes more linear. An interesting observation is that the training accuracy goes up to around 50% after the first mini-batch has been processed. An explanation could be that after the first backward pass, the segmenter changes its weights from a random state to a state based upon this first mini-batch. Afterwards, the training accuracy is only evaluated on this first mini-batch. This also explains why the test accuracy starts to increase later on, as this test set contains only novel images and therefore needs a more general model.

Epoch	Train Accuracy (%)	Test Accuracy (%)
0 (Initial)	16.134	16.24
0.05 (Mini-batch 1)	50.781	16.24
0.15 (Mini-batch 41)	54.573	24.32
0.2 (Mini-batch 61)	57.275	44.20
0.25 (Mini-batch 81)	59.751	48.46
0.35 (Mini-batch 121)	64.689	59.92
0.45 (Mini-batch 161)	68.323	62.89
0.5 (Mini-batch 181)	69.782	64.58
0.6 (Mini-batch 221)	71.854	65.07
1.0 (Mini-batch 381)	77.227	69.94
1	77.472	69.92
2	86.442	70.03
3	87.510	72.67
10	88.776	74.27
20	89.316	74.41

TABLE II

TRAINING AND TEST ACCURACY AT SELECTED EPOCHS FOR AN FSCNN MODEL

The FSCNN model in this research uses a frozen CNN model with a test accuracy of 77%. This means that adding the segmenter onto the existing model decreases the accuracy. This reduction appears to be permanent, as the FSCNN accuracy remains around 74-75% over a larger number of epochs, as shown in Table II. To avoid overfittting, the FSCNN after epoch 3 (accuracy = 72.67%) is chosen. This means that to create the FSCNN model, which aims to explain the original CNN model, some predictive accuracy is lost.

3.2. *Visual comparison*

Figure 3 provides a visualization of the training effects on the SCNN model, specifically focusing on the segmented images produced by the segmenter.

An interesting observation is the correlation between accuracy and the visual quality of the segmented images. At lower epochs, when the accuracy is low, the images appear blurry. By epoch 7, the images become more distinct, and the accuracy significantly improves. After this point, the accuracy increases minimally, and there is little visual difference in the segmented images. This indicates that overfitting does not appear to be an issue for the segmented images, maintaining consistency in both accuracy and visual clarity.

The consistency of the segments across different images is notably high. This suggests that while the segmenter maintains stable segments, it mainly refines the delineation of these segments during training.

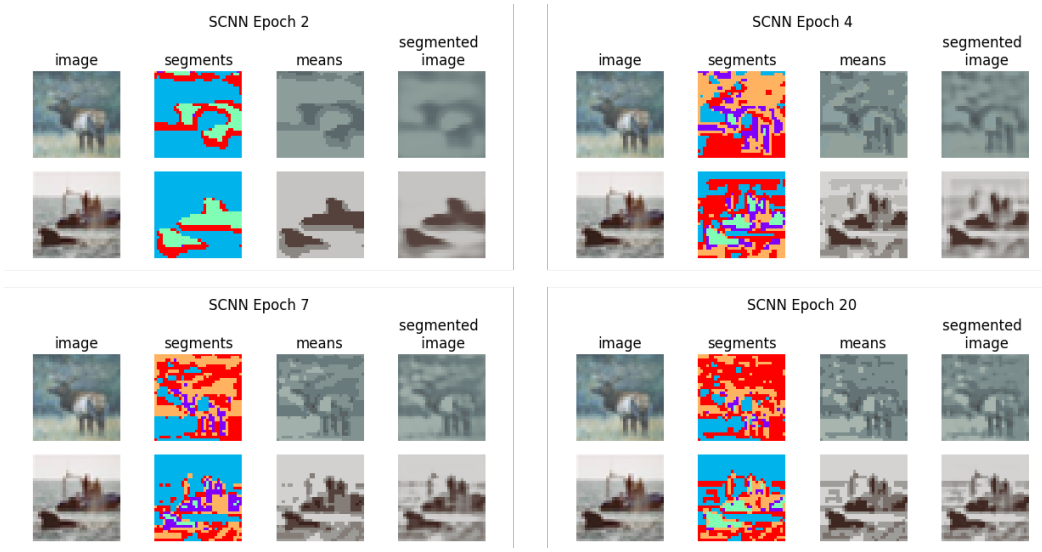

FIGURE 3.—SCNN: Visualisations of the segmenter while training.

With regards to the FSCNN, it becomes clear that the segmenter is capable of creating clear images early in the training process. This aligns with the high initial accuracy observed in Table II. Visualisation of the segmenter of the FSCNN model in training can be found in Figure 4.

Both the SCNN and the FSCNN produce similar images when adequately trained. In terms of consistency and behavior across epochs, the two models exhibit comparable performance. However, a notable difference emerges when considering the size of the segments. The images generated by the FSCNN's segmenter tend to produce larger and more distinct segments. Additional visual evidence of this can be found in the appendix.

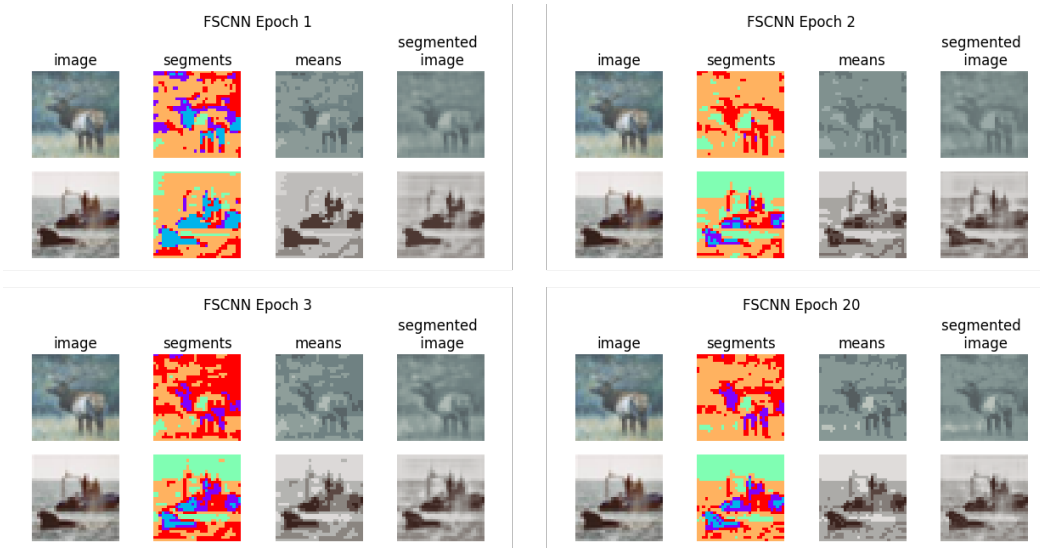

FIGURE 4.—FSCNN: Visualisations of segmenter while training.

3.3. *Comparison of segment sizes*

This section examines the size of the segments produced by the SCNN and FSCNN model. Segments are created by first using the segmentation model to process a batch of images, assigning a probability to each segment for every pixel in the images. For each pixel, the segment with the highest probability is selected, effectively labeling each pixel with a segment. The size of each segment is then computed by counting the number of pixels assigned to each segment label within each image. This process is repeated for all images in the batch. As the number of segments is set to 5 and the test set contains 10.000 images, the total number of created segments is around 50.000. The reason the number of segments is not exactly 50.000 is because the segmenter can choose to assign 0 pixels to a class, thereby effectively eliminating a segment.

As shown in Figure 5, there is a distinct difference between the SCNN and FSCNN models in terms of segment sizes. The SCNN model predominantly produces medium-sized segments, with a notable concentration in the 100-200 pixel range.

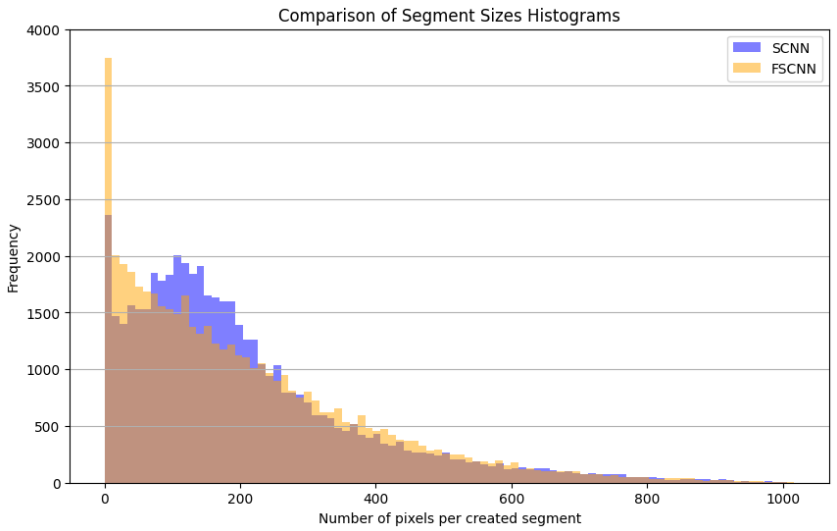

FIGURE 5.—Histogram of the segment sizes for the SCNN and FSCNN model.

In contrast, the FSCNN model generates a wider variety of segment sizes, including both very small and larger segments. This claim is supported by the tables in the subsection for segment sizes in the Appendix. Figure 5 illustrates that the variance in segment sizes for the FSCNN is greater than that of the SCNN. Another interesting observation is the presence of segments with zero pixels, indicating that the model chose not to assign any pixels to a specific segment. The count of segments with pixel size 0 is 645 for the SCNN model and 1061 for the FSCNN Model. Therefore, the FSCNN model chooses more often to not use all possible segments for the segmentation.

3.4. *Segmentation-based salience maps*

This section introduces and assesses segmentation-based heatmaps as a possible post-hoc explainability method, thereby tackling the Black Box problem. Heatmaps are widely used in computer vision as a method to visualize the regions of an image that are most influential in the decision-making process of a neural network. They aim to provide an intuitive way to understand which parts of the input contribute most to the model's predictions.

Figure 6 demonstrates the visualizations of the original image and the corresponding heatmaps using vanilla gradient, guided backpropagation and simple pixel perturbation. These images have been created using a CNN model with an accuracy of 0.77, corresponding to the model in 3.1 with 7 epochs of training.

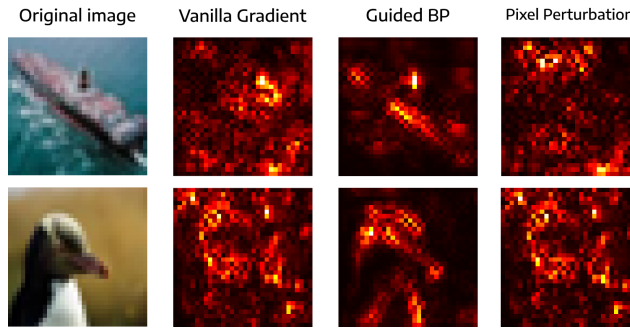

FIGURE 6.—Salience maps using different heatmapping techniques.

The vanilla gradient heatmap is created by computing the gradient of the output with respect to the input image. This gradient highlights the areas where small changes would most affect the output, thereby indicating the importance of each pixel. In practice, this involves backpropagating the gradients from the output layer to the input layer and visualizing the magnitude of these gradients.

Guided backpropagation, on the other hand, is a refinement of the vanilla gradient method. It modifies the backpropagation process by discarding gradients flowing backward through non-positive activations in the ReLU layers. This approach results in cleaner and more focused heatmaps by removing noise and highlighting only the most relevant parts of the image.

The simple perturbation heatmap is made by altering (perturbing) each pixel in the input image and observing the effect on the model's output. This method is different from the previous two as it is not gradient-based but perturbation-based (Nielsen et al. (2022)). The created heatmap resembles closely to the vanilla gradient method for these images at the chosen level of perturbation ($\epsilon = 1e - 5$).

However, the heatmapping methods in Figure 6 show signs of deficiency. A significant problem is that Vanilla Gradient and simple Pixel Perturbation produce noisy, highlighting irrelevant features, and less interpretable heatmaps. The Guided Backpropagation method produces a less noisy heatmap, but Adebayo et al. (2020) have shown that this technique can be insensitive to the model's parameters.

The problem all three heatmaps have is the granular manner in which they have to be interpreted. Human beings don't seem to consider and interpret visual data pixel by pixel. Rather, it seems as if the human brains constructs larger shapes and objects within a scene. This segmentation process aligns with our natural cognitive ability to simplify complex visual information into meaningful chunks, making it easier to process and interpret what we see.

To tackle the current heatmapping deficiencies and to account for this more human-like way of processing visual data, segmentation-based heatmaps are introduced. Figure 7 shows two original images, the created segments and segmentation-based heatmaps for different levels of perturbation (ϵ). The lighter - mainly white and yellow - the segment is, the more salient it is.

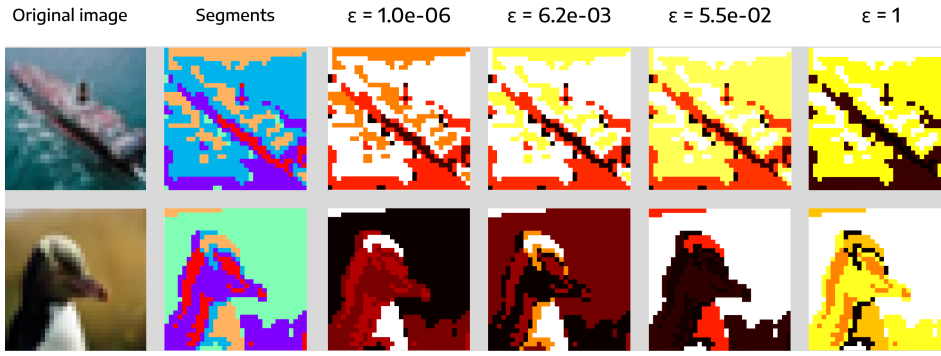

FIGURE 7.—Visualisation of the original image, created segments and segmentation-based heatmaps for different values of epsilon.

The segments in Figure 7 are created by an FSCNN with an accuracy of 0.73, corresponding to the model in 3.1 with 3 epochs of training. Afterwards, the segmentation-based salience maps are created by calculating the salience for each segment q in the image, using the difference in the model's output before and after a small perturbation is applied to the segment. The complete mathematical process is presented in 2.3. The segmentation-based heatmaps are perturbation-based and as the FSCNN hasn't changed the original CNN's parameters, these heatmaps are a post-hoc explainability methods.

Conducting research into this technique has produced several insights.

Firstly, Figure 7 shows that the assigned salience for the segments changes according to the level of perturbation. In the case of the bird (bottom picture), it seems that the background becomes more salient as epsilon increases. For the upper picture, the visual assessment is harder as the created segments themselves do not seem to correspond to human-like interpretation. More visualisation are presented in the Appendix. In summary, both the level of epsilon and the created segments are important in the visualisation of the segmentation-based heatmap.

Secondly, the variance in salience across segments changes for the level of perturbation. This finding is illustrated in Figure 8. For small epsilons, the standard deviation of the salience is stable but for larger epsilon, $(0.23 < \epsilon < 1)$, it drops. Figure 8 has been made by calculating the salience for all created segments in the test data, which is around 50.000. An explanation for this could be that using (too large) perturbations changes the segments to such an extent that significant differences in salience disappear. For interpretability, this likely means that the salience maps become less meaningful, as it is the difference in salience that provide us with better understanding of the model.

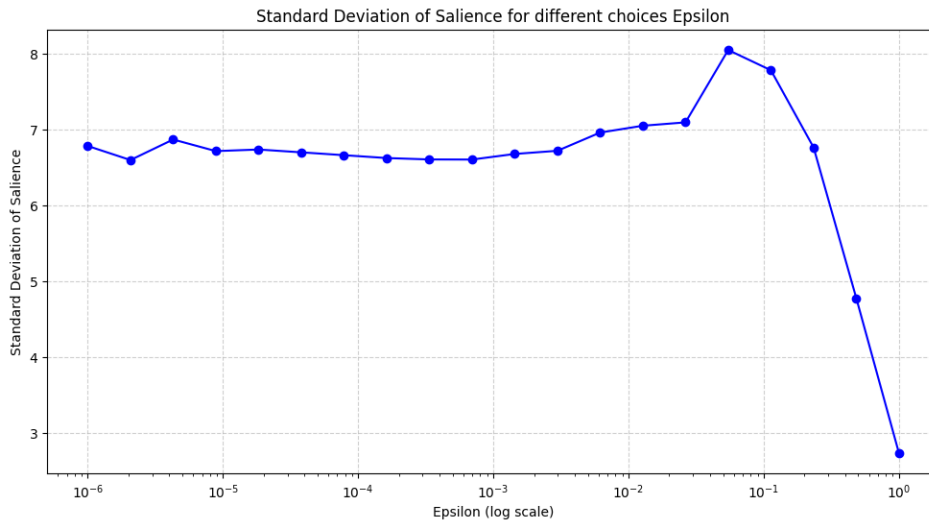

FIGURE 8.—Standard deviation of the salience for all the created segments in the test data.

Thirdly, segmentation-based perturbation shows great improvement over simple pixel perturbation in terms of computational efficiency. The calculation for the salience has shown to be approximately 400 times faster for the segmentation-based method. This result was found by calculating the salience of all segments for 128 images using the T4 GPU from Google Colab. The table with computing times is illustrated in the Appendix.

3.5. *Philosophical implication of segmentation-based heatmaps*

The use of segmentation-based heatmaps to explain intricate model decisions has important philosophical implications. Through emphasizing the most consequential input characteristics, these heatmaps address key epistemic needs for different agents interacting with an AI model.

Segmentation-based heatmaps, which focus on defining the input-output transformations and the reasons behind these changes, efficiently handle both "what" and "why" issues at the computational level by utilizing Zednik's framework (Zednik (2021)).

In a segmented approach, likely closer to human perception, segmentation-based heatmaps determine the input elements primarily responsible for a system's output. In doing so it specifies the transition function $f(x)$ and 'What' questions are therefore answered. Furthermore, these heatmaps address "why" questions by highlighting the areas of a picture that are essential for classification, as illustrated in the bird example in Figure 7.

Creators, such as system administrators and software engineers, might learn more about how the models function, enabling bias correction and fine-tuning. Creators can detect components that are over- or under-emphasized by knowing which features influence the model's judgments. This information may enable more precise adjustments to the model by the creator.

Based on the model's advice (output), executors, such as back-office staff or regulatory officials, could make more informed decisions. This is due to the fact that segmentation-based heatmaps might improve comprehension of the precise input elements that are crucial to the result. A deeper understanding could increase societal confidence and increase the adoption of sophisticated AI systems.

But depending solely on XAI methods such as heatmaps has inherent limitations and can be risky, as Rudin et al. (2022) points out. This is an important philosophical consideration. Interpretable models, as opposed to described black-box models, should be employed for high-stakes decisions, according to Rudin et al. (2022), as the latter might be deceptive and encourage misplaced faith. Although helpful, heatmaps don't entirely describe how features are utilized to make decisions. Rudin et al. (2022) compares this to a real estate agent saying, without going into detail, that the roof and backyard determine a house's price. This comparison demonstrates that while heatmaps increase interpretability, in high-stakes settings they may not be sufficient to completely explain a model.

In conclusion, segmentation-based heatmaps are a promising method in Explainable AI, fulfilling certain epistemic needs. However, they are not capable of fully explaining the model's decision process.

4. CONCLUSION

This thesis has investigated the use of Segmentation-based heatmaps as a post-hoc explainability method to address the Black Box problem within computer vision. By integrating a segmentation model with a CNN, both a Segmentation-based CNNs (SCNNs) and Frozen Segmentation-based CNNs (FSCNNs) are introduced. The FSCNN model is used to segment images in a post-hoc manner, so that the corresponding heatmaps serve as an explainability method.

The three models show different behavior during training. The CNN consistently outperforms the SCNN in terms of test accuracy by approximately 5-6%. Notably, the FSCNN model shows quick initial learning but then ends up at a slightly lower accuracy than the original CNN.

The visual assessment of the segmented images indicates that both SCNN and FSCNN models produce consistent and meaningful segments once adequately trained. However, the FSCNN tends to generate larger segments compared to the SCNN.

The proposed segmentation-based heatmaps seem to show promising improvements over traditional heatmapping techniques. Visual assessment of these heatmaps reveals that the level of perturbation (ϵ) affects the salience of segments. Additionally, the standard deviation of salience values remains stable for small perturbations but decreases for larger ones. This finding demonstrates the importance of selecting appropriate perturbation levels to preserve the explanatory power of the heatmaps. Moreover, the segmentation-based perturbation methods demonstrates substantial computational efficiency, being approximately 400 times faster than simple pixel perturbation. This finding is noteworthy, as computational efficiency is a great problem for perturbation-based heatmaps (Nielsen et al. (2022)).

Segmentation-based heatmaps address key epistemic needs by highlighting influential input features, effectively answering 'what' and partially 'why' questions at the computational level (Zednik (2021)). The segmentation-based heatmaps can thereby provide valuable insights for different agents interacting with the model. However, as noted by Rudin et al. (2022), heatmaps may fall short in fully explaining decision processes.

In conclusion, this thesis presents segmentation-based heatmaps as a promising approach for post-hoc explainability within computer vision. By integrating segmentation with CNNs, the FSCNN model is able to enhance the interpretability of the original model without drastically decreasing the accuracy.

5. DISCUSSION

Although the segmentation-based heatmaps seem promising, much further research is required to assess their true potential.

First of all, the effect of different implementational choices on the model's output can be considered. There are many different parameters that can be changed, such as the configuration of the optimization algorithm. A great reason to do so would be to improve the segmenter's accuracy and therefore the quality of the segments. As noted in the results, the segmentation-based heatmaps are restricted by the created segments. A better-performing segmenter could mean the segmentation-based heatmaps are less noisy and therefore more interpretable.

Secondly, this research has restricted itself to choosing 5 segments. Considering a vaster range of segments would increase the extensiveness of this research. Another interesting idea is to give the segmenter more freedom to choose the amount of segments based on the input image. This seems as a necessary step to improve the segmentation-based heatmaps, as it can be assumed that different images can be divided into a different optimal amount of segments.

Thirdly, a different choice of dataset could lead to new findings as the CIFAR-10 dataset consists of low resolution images. Therefore, it would be interesting to see how the segmentation-based heatmaps respond to higher resolution images. Additionally, a dataset containing more than one identifiable objects could also produce novel insights.

Lastly, the Black Box problem might need to be approached in a completely different manner. As pointed out by Rudin et al. (2022), post-hoc methods seem to fall short. Another interesting observation for segmentation-based heatmap specifically is that the segmenter, which enables the creation of the heatmaps, is a CNN itself. This seems as if a Black Box is used to explain a Black Box. The potential epistemic problem following this observation is left for further inquiry.

REFERENCES

Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M., and Kim, B. (2020). Sanity checks for saliency maps. [5, 18]

Beisbart, C. and Räz, T. (2022). Philosophy of science at sea: Clarifying the interpretability of machine learning. *Philosophy Compass*, 17(6):e12830. [3]

Cian, D., van Gemert, J., and Lengyel, A. (2020). Evaluating the performance of the lime and grad-cam explanation methods on a lego multi-label image classification task. [4]

Goodman, B. and Flaxman, S. (2017). European union regulations on algorithmic decision making and a “right to explanation”. *AI Magazine*, 38(3):50–57. [3]

Long, J., Shelhamer, E., and Darrell, T. (2015). Fully convolutional networks for semantic segmentation. [5]

Minaee, S., Boykov, Y., Porikli, F., Plaza, A., Kehtarnavaz, N., and Terzopoulos, D. (2020). Image segmentation using deep learning: A survey. [5]

Nielsen, I. E., Dera, D., Rasool, G., Ramachandran, R. P., and Bouaynaya, N. C. (2022). Robust explainability: A tutorial on gradient-based attribution methods for deep neural networks. *IEEE Signal Processing Magazine*, 39(4):73–84. [4, 5, 17, 21]

O’shea, K. and Nash, R. (2015). An introduction to convolutional neural networks. [4]

Ribeiro, M. T., Singh, S., and Guestrin, C. (2016). "why should i trust you?": Explaining the predictions of any classifier. [4]

Rudin, C., Chen, C., Chen, Z., Huang, H., Semenova, L., and Zhong, C. (2022). Interpretable machine learning: Fundamental principles and 10 grand challenges. *Statistic Surveys*, 16:1–85. [3, 20, 21, 22]

Zednik, C. (2021). Solving the black box problem: A normative framework for explainable artificial intelligence. *Philosophy & Technology*, 34:265–288. [3, 10, 20, 21]

6. APPENDIX

6.1. *Training and Test Accuracy*

Epoch	Train Accuracy (%)	Test Accuracy (%)
1	14.534	30.44
2	39.55	51.05
3	56.818	62.02
4	68.228	69.88
5	75.682	74.98
6	81.142	75.11
7	85.788	77.89
8	89.438	76.55
9	92.474	75.69
10	93.996	76.48
11	95.3	78.05
12	96.354	76.05
13	96.852	77.91
14	97.334	77.78
15	97.518	78.21
16	98.11	78.03
17	98.236	77.87
18	98.372	78.49
19	98.374	77.7
20	98.674	79.05

TABLE III

TRAINING AND TEST ACCURACY OVER 20 EPOCHS FOR A CNN MODEL

Epoch	Train Accuracy (%)	Test Accuracy (%)
1	14.256	20.06
2	29.376	34.75
3	43.946	44.56
4	55.54	58.18
5	64.876	63.27
6	72.23	69.74
7	77.852	68.86
8	82.584	71.36
9	86.332	71.93
10	89.304	72.57
11	91.832	71.11
12	93.05	72.17
13	94.38	71.15
14	95.166	71.3
15	95.846	72.95
16	96.274	72.53
17	96.492	72.59
18	97.312	72.04
19	97.154	73.42
20	97.528	73.14

TABLE IV

SCNN TRAINING AND TEST ACCURACY PER EPOCH

Epoch	Train Accuracy (%)	Test Accuracy (%)
0 (Initial)	16.134	16.24
0.05 (Mini-batch 1)	50.781	16.24
0.1 (Mini-batch 21)	54.018	16.26
0.15 (Mini-batch 41)	54.573	24.32
0.2 (Mini-batch 61)	57.275	44.20
0.25 (Mini-batch 81)	59.751	48.46
0.3 (Mini-batch 101)	62.376	54.72
0.35 (Mini-batch 121)	64.689	59.92
0.4 (Mini-batch 141)	66.550	60.94
0.45 (Mini-batch 161)	68.323	62.89
0.5 (Mini-batch 181)	69.782	64.58
0.55 (Mini-batch 201)	70.931	65.09
0.6 (Mini-batch 221)	71.854	65.07
0.65 (Mini-batch 241)	72.760	68.07
0.7 (Mini-batch 261)	73.599	67.60
0.75 (Mini-batch 281)	74.324	68.73
0.8 (Mini-batch 301)	75.091	69.35
0.85 (Mini-batch 321)	75.725	69.72
0.9 (Mini-batch 341)	76.299	69.87
0.95 (Mini-batch 361)	76.764	65.65
1.0 (Mini-batch 381)	77.227	69.94
1	77.472	69.92
2	86.442	70.03
3	87.510	72.67
4	87.606	72.23
5	88.128	72.95
6	88.296	72.04
7	88.316	73.71
8	88.500	74.00
9	88.754	73.78
10	88.776	74.27
11	88.694	74.28
12	88.830	74.61
13	88.832	74.50
14	89.124	74.36
15	89.034	74.40
16	88.920	74.60
17	89.248	74.66
18	89.152	74.42
19	89.176	74.88
20	89.316	74.41

TABLE V

TRAINING AND TEST ACCURACY OVER 20 EPOCHS FOR AN FSCNN MODEL, INCLUDING DETAILED MINI-BATCH STEPS FOR EPOCH 0

6.2. *Visualisations of segmenter while training*

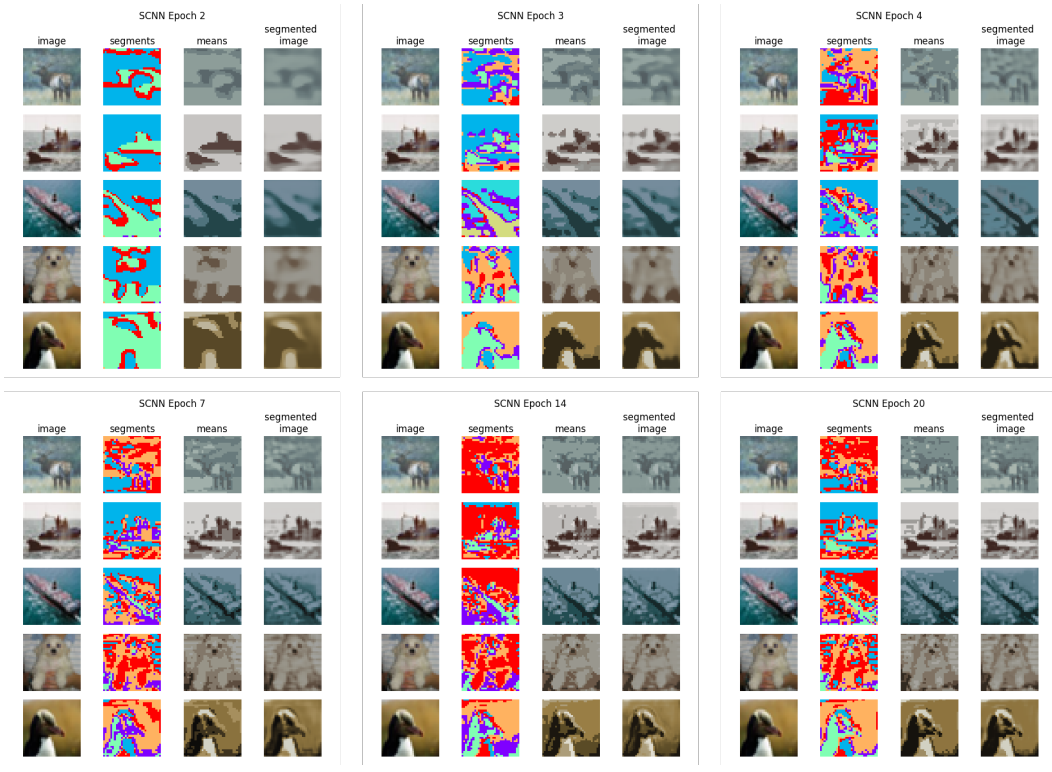

FIGURE 9.—The segmented images for a SCNN model.

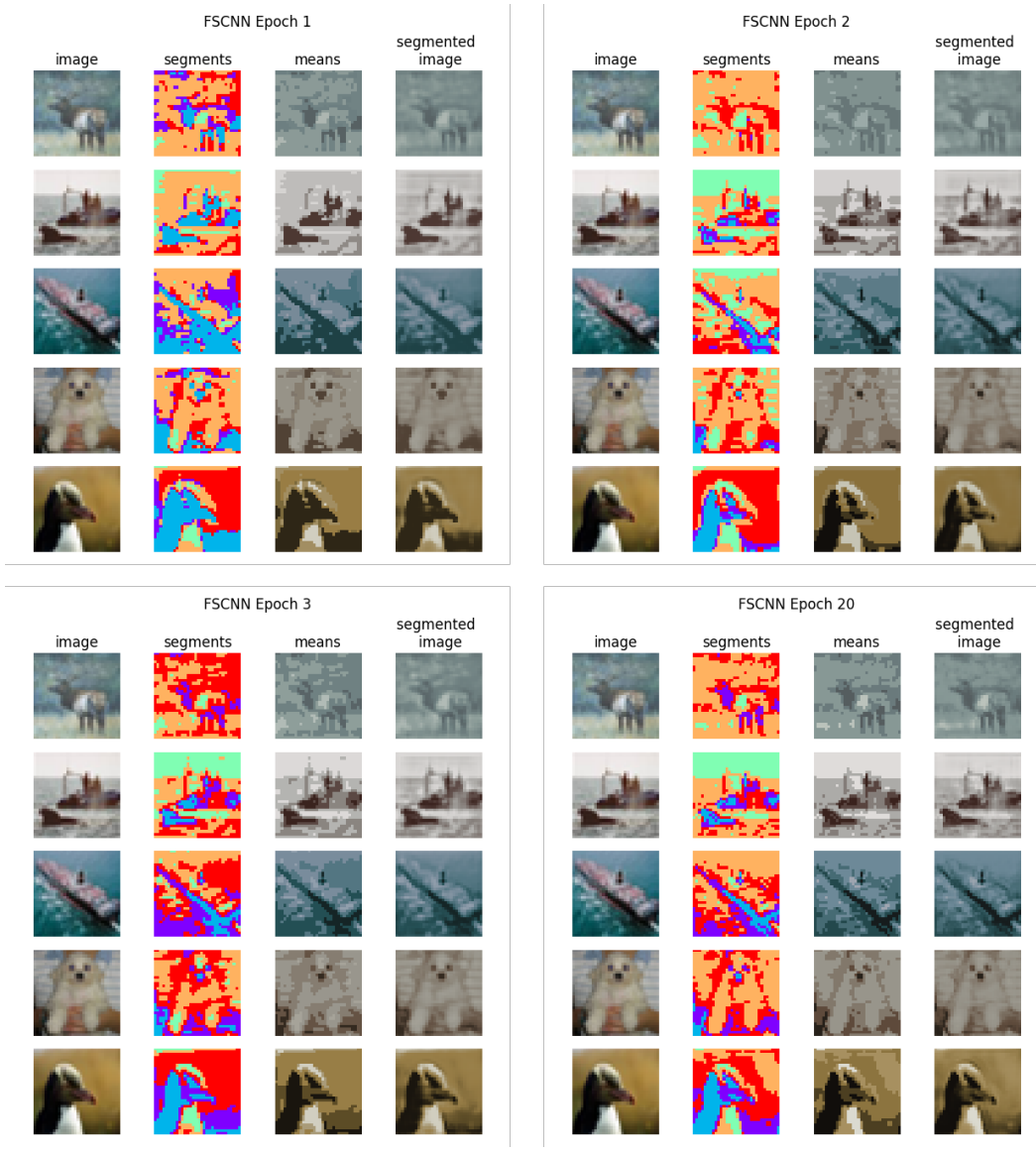

FIGURE 10.—The segmented images for a FSCNN model.

6.3. *Segment sizes*

Description	**SCNN**	**FSCNN**
Total amount of segments	50000	50000
Mean segment size	204.8	204.8
Median segment size	161.0	159.0
Standard deviation of segment sizes	171.144	179.065
Minimum segment size	0	0
Maximum segment size	1018	1017

TABLE VI
SEGMENT STATISTICS FOR SCNN AND FSCNN

Description	**SCNN**	**FSCNN**
Number of observations with segment size 0	645	1061
Number of observations with segment size above 500	6308	7041

TABLE VII
COMPARISON OF OBSERVATION COUNTS FOR SCNN AND FSCNN

Description	**SCNN**	**FSCNN**
Mean segment size	207.476	209.240
Median segment size	163.0	164.0
Standard deviation of segment sizes	170.639	178.411
Minimum segment size	1	1
Maximum segment size	1018	1017

TABLE VIII
FILTERED (SEGMENT SIZE = 0) SEGMENT SIZES ANALYSIS FOR SCNN AND FSCNN

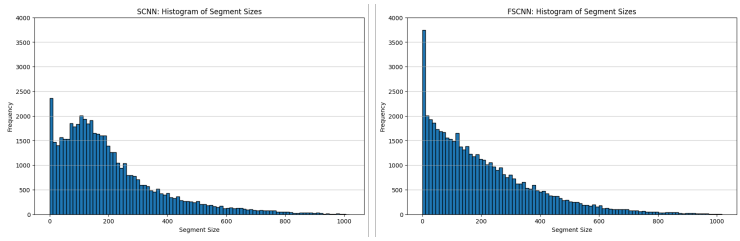

FIGURE 11.—Histograms of the produced segment sizes for the SCNN (left) and the FSCNN model (right).

6.4. *Segmentation-based Heatmaps*

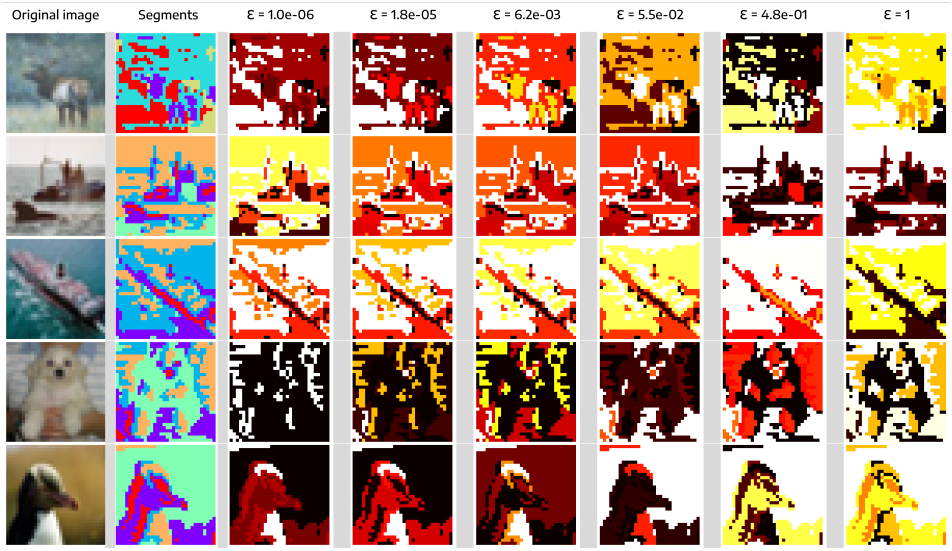

FIGURE 12.—Visualisation of the segmentation-based heatmaps for 5 images and 6 different values of epsilon.

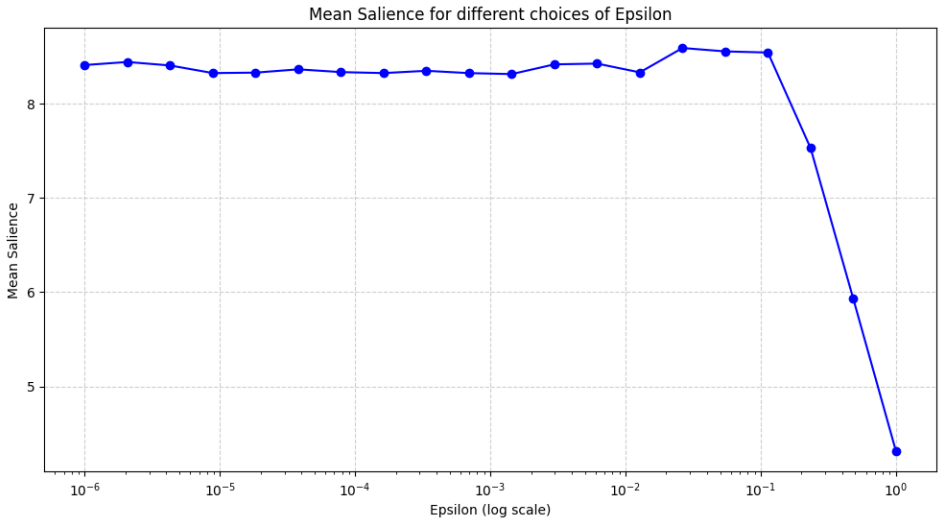

FIGURE 13.—Mean Salience for different choices of Epsilon.

Number of Images	Pixel-wise Perturbation Time (s)	Segmentation-based Perturbation Time (s)
5	23.98	0.05
10	45.13	0.09
15	65.73	0.14
20	87.36	0.18
25	108.41	0.22
30	129.63	0.27
35	150.11	0.31
40	171.19	0.35
45	193.09	0.39
50	214.31	0.44
55	234.88	0.48
60	256.05	0.52
65	277.22	0.56
70	298.48	0.60
75	319.13	0.65
80	340.10	0.71
85	361.58	0.77
90	382.96	0.83
95	404.05	0.90
100	424.62	0.97
105	445.79	1.02
110	466.78	1.08
115	487.93	1.13
120	508.32	1.19
125	529.47	1.24
Total Batch Time (128 images)	**542.07 s**	**1.27 s**

TABLE IX

COMPUTATION TIME FOR PIXEL-WISE PERTURBATION AND SEGMENTATION-BASED PERTURBATION SALIENCE MAPS FOR FSCNN WITH EPSILON EQUAL TO 1E-3

6.5. *Architect and training setup for the CNN model and segmenter*

The tables below detail the architectures of the CNN and Segmenter models. Both models incorporate convolutional layers, with the CNN designed for classification and the Segmenter for image segmentation. The activation functions used in these layers play crucial roles in their respective functions.

In the CNN model, ReLU (Rectified Linear Unit) activation functions follow each convolutional and fully connected layer.

The Segmenter model employs both ReLU and Softmax activation functions. ReLU is used after the first two convolutional layers. The final convolutional layer in the Segmenter model uses a Softmax activation function.

Layer	Layer Type	Configuration	Output Shape
Conv Layer 1a	Conv2d	in_channels=3, out_channels=32, kernel_size=3, stride=1, padding=1	32 x 32 x 32
Conv Layer 1b	Conv2d	in_channels=32, out_channels=64, kernel_size=3, stride=1, padding=1	64 x 32 x 32
Pooling Layer 1	MaxPool2d	kernel_size=2, stride=2	64 x 16 x 16
Conv Layer 2a	Conv2d	in_channels=64, out_channels=128, kernel_size=3, stride=1, padding=1	128 x 16 x 16
Conv Layer 2b	Conv2d	in_channels=128, out_channels=128, kernel_size=3, stride=1, padding=1	128 x 16 x 16
Pooling Layer 2	MaxPool2d	kernel_size=2, stride=2	128 x 8 x 8
Conv Layer 3a	Conv2d	in_channels=128, out_channels=256, kernel_size=3, stride=1, padding=1	256 x 8 x 8
Conv Layer 3b	Conv2d	in_channels=256, out_channels=256, kernel_size=3, stride=1, padding=1	256 x 8 x 8
Pooling Layer 3	MaxPool2d	kernel_size=2, stride=2	256 x 4 x 4
Flatten Layer	Flatten	-	4096
Fully Connected Layer 1	Linear	in_features=4096, out_features=1024	1024
Fully Connected Layer 2	Linear	in_features=1024, out_features=512	512
Output Layer	Linear	in_features=512, out_features=10	10

TABLE X

ARCHITECTURAL CONSIDERATIONS FOR THE CNN MODEL

Layer	Layer Type	Configuration	Output Shape
Conv Layer 1	Conv2d	in_channels=3, out_channels=16, kernel_size=5, stride=1, padding=2	16 x 32 x 32
Conv Layer 2	Conv2d	in_channels=16, out_channels=16, kernel_size=5, stride=1, padding=2	16 x 32 x 32
Conv Layer 3	Conv2d	in_channels=16, out_channels=5, kernel_size=5, stride=1, padding=2	5 x 32 x 32

TABLE XI

ARCHITECTURAL CONSIDERATIONS FOR THE SEGMENTER MODEL

Training Setup	Description
Loss Function	CrossEntropyLoss
Optimizer	SGD (lr=0.1, momentum=0.7)
Batch Size	128
Number of Epochs	7
Device	cuda (if available)
Dataset	CIFAR-10
Transformations	ToTensor, Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))

TABLE XII

TRAINING SETUP FOR THE CNN AND SCNN MODEL